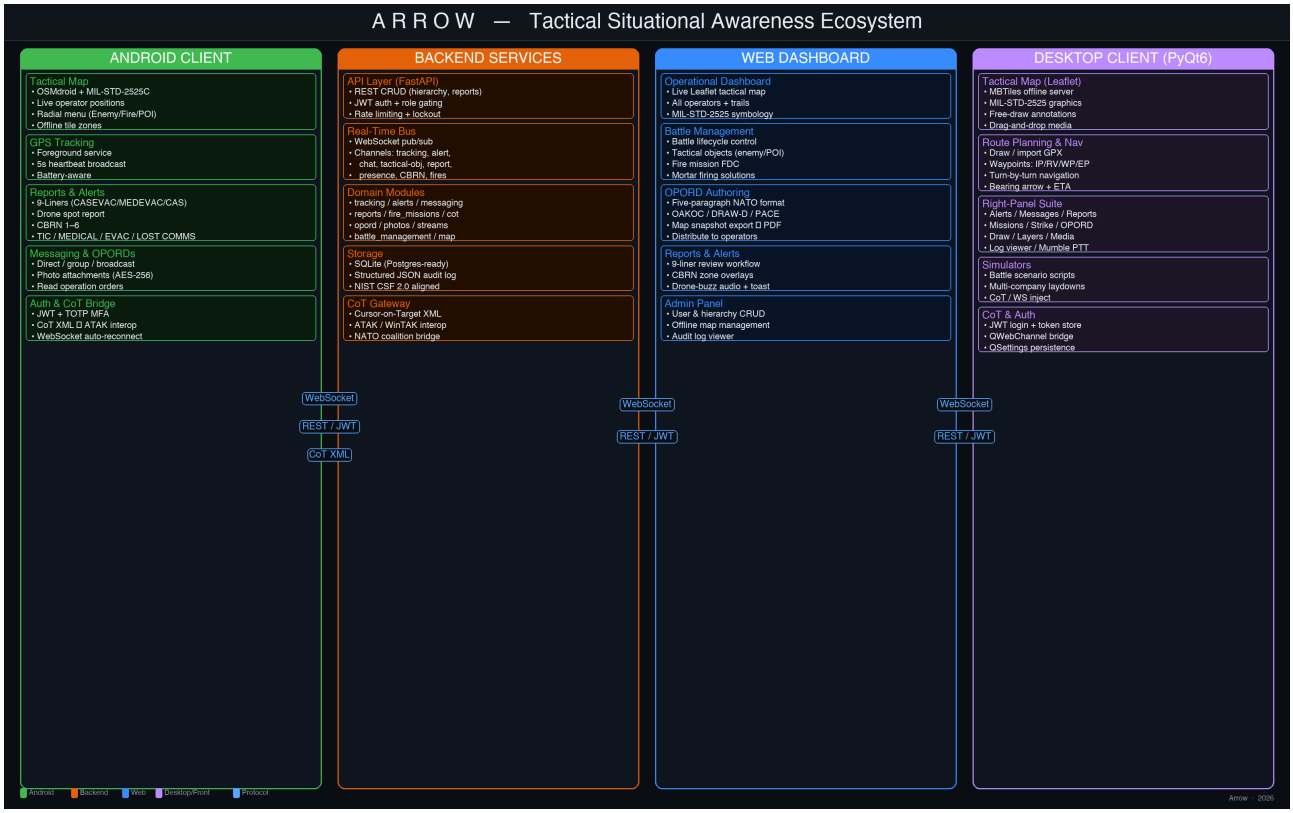
A R R O W

Open Situational Awareness for the Warfighter



What Arrow Is and Why It Exists

Modern tactical operations produce an enormous, continuous flood of information. GPS positions, contact reports, fire missions, medical evacuations, chemical hazard zones, drone observations, operational orders — all of it arrives simultaneously, from multiple echelons, across multiple networks, through multiple device types. The traditional answer to this challenge has been fragmentation: one application for the map, another for chat, a paper form for the 9-liner, a radio call for the contact report, a PDF for the operation order. The result is a "common operating picture" that is rarely common and rarely current.

Arrow exists to solve exactly that problem. It is a TAK-class situational awareness platform — open, fully auditable, and deployable on infrastructure the unit controls — designed to give every element from individual operator to company commander a single, unified, real-time picture of the battlefield. No black boxes. No vendor lock-in. No licence that expires mid-rotation.

The platform is built around one central principle: every event that matters on a tactical network — a position update, a contact report, an emergency alert, a fire mission, a medical request, a CBRN observation — should reach every person who needs it, immediately, on whatever device they are carrying, without any manual hand-off between systems.

The Four Pillars of the Arrow Ecosystem

Arrow is not a single application. It is a four-component ecosystem: a backend services layer, an Android tactical client, a browser-based operational dashboard, and a PyQt6 desktop client. Each component is built for a distinct context and user, but they share a single data model, a single authentication layer, and a single real-time communications bus. Changing a tactical object on any one client propagates to all others in under a second.

These four pillars are described in detail below, followed by the communication protocols that bind them together.

Backend Services — The Operational Core

The backend is the authoritative source of truth for the entire Arrow network. It is built on FastAPI, chosen for its native asynchronous support, automatic OpenAPI documentation, and clean dependency injection model. Authentication uses JWT tokens with a 256-bit auto-generated secret; every endpoint that modifies state requires a valid token, and admin and Battle Captain endpoints are additionally gated by role (ADMIN, BATTLE_CAPTAIN, OPERATOR). Optional TOTP multi-factor authentication adds a second factor for privileged accounts.

Domain logic is organised into discrete, pluggable modules, each with its own APIRouter: tracking (live GPS persistence and broadcast), alerts (one-tap emergency broadcasts), reports (structured tactical forms including 9-liners, drone spot, and CBRN), messaging (direct, group, and broadcast chat), fire_missions (call-for-fire with mortar firing solutions), opord (five-paragraph NATO operation order authoring and distribution), photos (AES-256-GCM encrypted media storage), streams (live MJPEG camera feeds from operators to the TOC), battle_management (battle lifecycle gated to Battle Captain and Admin roles), and cot (Cursor-on-Target XML encoding and decoding for ATAK interoperability).

All persistent state lives in SQLite with a schema that is ready to migrate to PostgreSQL when the unit scales. Every action that modifies state produces a structured JSON audit log entry, aligned to NIST Cybersecurity Framework 2.0 categories (DE.CM / GV.OV). Rate limiting, account lockout on repeated failed logins, and token revocation via an in-memory blacklist (optionally Redis-backed) round out the security posture.

A single in-process WebSocket pub/sub broadcaster fans every real-time event across all connected clients. The broadcaster interface is deliberately clean and pluggable: swap the in-process implementation for Redis or NATS without touching a single caller. Active channels include: tracking, tactical-object, alert, chat, report, presence, cbrn, and fires.

Android Client — The Operator's Device

Operators in the field run the Arrow Android application, built entirely in Kotlin with Jetpack Compose for the UI. The application is designed to scale seamlessly between phone and tablet form factors in both portrait and landscape orientations, using a WindowSize class-based layout system.

At the centre of the Android experience is the tactical map, powered by OSMdroid with full MIL-STD-2525C military symbology. Every connected operator appears as a blue friendly icon; enemy contacts appear as red; unknowns as yellow; neutrals as green. A foreground GPS tracking service runs continuously, broadcasting position every five seconds to the entire network via WebSocket — the

heartbeat that keeps every map current. Offline tile zones can be downloaded in advance, allowing the application to maintain full map functionality in degraded-connectivity environments.

From the radial menu — a tap-and-hold gesture on the map — operators can drop tactical objects: enemy contacts, fire mission markers, points of interest, mortar positions, drone spot markers, and measurement anchors. The measure tool computes distance and initial bearing (in degrees and NATO mils) between any two map points using spherical-earth mathematics.

The reporting suite covers the full tactical cycle. Operators submit contact and spot reports with location, size, direction, and threat assessment. They file 9-liner CASEVAC, MEDEVAC, and CAS requests in structured digital form — eliminating radio transcription errors. The drone spot module captures type (QUADCOPTER, FPV, LOITERING_MUNITION, SHAHED-136, MAVIC, ISR, FIXED_WING), estimated altitude, direction of travel, speed, and behavior in a single structured submission that simultaneously creates a report record and raises a DRONE_SPOTTED alert across the network. NATO CBRN 1–6 reports following STANAG 2103 / ATP-45 can be submitted as structured forms.

Emergency alerts — Troops in Contact, Medical, EVAC, Lost Communications — are a single tap broadcast with audio, visual escalation, and a map marker that appears instantly on every connected display. Operation orders authored by the Battle Captain are readable in-app. Messaging supports direct, group, and broadcast chat with photo attachments stored encrypted at rest (AES-256-GCM). Authentication uses JWT with optional TOTP MFA; the CoT bridge enables position and event sharing with ATAK and other TAK-ecosystem clients on the same coalition network.

Web Dashboard — The Battle Captain's Screen

Battle Captains and Admins access Arrow through a browser-based Flask operational dashboard. This is the digital operations room — a single screen that replaces a wall of maps, radio logs, and whiteboards.

The tactical map at the centre of the dashboard shows every operator's live position, trail, callsign, rank, status, team, section, and platoon. Tactical objects — enemy contacts, POIs, fire mission markers — are shared in real time with every other client. The Battle Captain can place objects using a right-click radial menu, move or delete them, and see CBRN hazard zone overlays (Zone I immediate hazard, Zone II downwind sector) automatically rendered when CBRN reports arrive.

The OPORD module brings doctrinal planning into the network. Battle Captains author complete five-paragraph NATO/US Operation Orders: Paragraph 1 Situation (OAKOC terrain analysis, civil dimension using ASCOPE, enemy DRAW-D assessment, EMLCOA/EMDCOA); Paragraph 2 Mission; Paragraph 3 Execution (commander's intent, CONOPS, subordinate tasks, CCIR, ROE, FSCM); Paragraph 4 Sustainment (supply classes, EPW handling, CASEVAC/MEDEVAC); Paragraph 5 Command and Signal (PACE plan, succession, challenge/password). Map snapshots stitched from OSM tiles with all tactical overlays embedded can be appended to the OPORD and the complete document exported as PDF or pushed directly to specific operator devices.

The Fire Direction Centre module receives calls for fire and computes L16 81mm mortar firing solutions — charge, quadrant elevation, deflection, and time of flight — live as the target position is adjusted. When emergency alerts arrive, the dashboard plays a distinctive audio tone, voices the alert type (using the Web Speech API), and shows a colour-coded persistent toast carrying all submitted detail. Incoming drone spot reports trigger a drone-buzz tone and a purple toast with every field. An admin panel provides full CRUD over the military hierarchy, offline map zone management, and the audit log viewer.

Desktop Client — Front (PyQt6)

The PyQt6 desktop client — referred to internally as 'front' — provides a native cross-platform tactical workstation for operators and battle captains running macOS, Windows, or Linux. It combines the full depth of the backend API with a high-fidelity local map experience that runs offline without any browser dependency.

The map is hosted inside a QWebEngineView rendering a Leaflet-based tactical map served by a local MBTiles tile server. A QWebChannel bridge connects Python signals and slots to JavaScript functions, giving Python code direct control over the map — placing and removing tracks, tactical objects, routes, free-draw overlays, KML/GPX layers, CBRN zones, and weather overlays — without any network round-trip. The bridge also receives events from the map back into Python: clicks, drawing completions, symbol selections, navigation waypoint arrivals.

Route planning is fully integrated: draw a route on the map or import a GPX file, assign waypoint types (Initial Point, Rendezvous, Waypoint, Endpoint), set a name and travel speed, and save. The system computes leg distances and estimated times. Routes persist across sessions via QSettings. Live navigation uses the browser's geolocation API to track GPS position, displays a bearing arrow pointing to the next waypoint, calculates remaining distance and ETA, auto-advances on arrival within 50 metres, and shows turn hints based on current heading.

The right-panel suite is a collapsible docked panel holding purpose-built subpanels: Alerts, Messages, Reports, Missions, Strike Packages, OPORD viewer, Draw tools, Layer manager, Media library, Log viewer, and integrated Mumble push-to-talk. MBTiles offline maps load directly from local files. The client connects to the backend via JWT-authenticated REST and WebSocket, using the same credentials and protocol as the Android and web clients.

How Arrow Communicates

Three complementary protocols bind the Arrow ecosystem together, each matched to what it carries.

WebSockets are the backbone of real-time communication. Every connected client — Android, web dashboard, and PyQt6 desktop — maintains a persistent WebSocket connection to the backend, authenticated by the same JWT token used for REST calls. The in-process pub/sub broadcaster fans every live event to all subscribers on the relevant channel with sub-second latency. Tracking updates, alert broadcasts, tactical object changes, chat messages, presence events, CBRN reports, fire mission updates, and stream notifications all flow through this single bus. The broadcaster interface is pluggable: an in-memory implementation handles single-server deployments; a Redis or NATS implementation drops in for distributed or high-availability configurations without any change to the callers.

REST APIs (FastAPI) handle everything structured and persistent: authentication and token issuance, CRUD operations on operators, hierarchy entities, tactical objects, reports, OPORDs, fire missions, and offline map manifests. Every mutating endpoint requires a valid JWT; role gating (ADMIN, BATTLE_CAPTAIN, OPERATOR) is enforced at the dependency level so no handler decodes tokens itself. Pydantic v2 schemas validate all inputs and serialise all outputs.

Cursor on Target (CoT) is the standard XML messaging protocol used by ATAK, WinTAK, and allied TAK-compatible systems. Arrow's CoT module encodes and decodes CoT XML, allowing Arrow operators to share positions and events with coalition partners running different clients on the same CoT-capable network — without a gateway, middleware, or per-partner integration project. This is the bridge to the broader NATO and partner-nation TAK ecosystem.

All traffic between clients and the backend passes through an nginx reverse proxy. HTTPS is available with either self-signed certificates (generated dynamically from server IPs at deploy time) or CA-signed certificates for production use. The backend port is not exposed to the host network — it is reachable only through the proxy — and the entire stack deploys with a single docker compose up -d.

Security and Deployment

Arrow is designed to run on infrastructure the unit controls, with no dependency on external services. The entire stack deploys from a single docker compose up -d command. The backend, web dashboard, MBTiles tile server, and nginx proxy run as separate containers on a defined internal network.

Authentication uses JWT with a 256-bit auto-generated secret, rotated on restart unless pinned in configuration. Account lockout activates after repeated failed login attempts. TOTP multi-factor authentication is available for all accounts and enforced for admin roles. Token revocation uses an in-memory blacklist backed optionally by Redis. Photo attachments on Android are encrypted at rest with AES-256-GCM before leaving the device. Every state- changing action produces a structured JSON audit log entry aligned to NIST CSF 2.0.

The platform is not a product with a vendor in the loop. The code is open, the tests pass, the protocols are standard, and every dependency is auditable. When the mission changes — new echelon, new reporting format, new interop partner — the platform changes with it.